Reporte de Eficiencia: Compactacion de Contexto y Uso de Tokens

Proyecto: Intrale Platform | **Fecha:** 2026-03-16 | **Sprint:** SPR-033

Problema: Las sesiones de Claude Code se compactan frecuentemente, perdiendo contexto y requiriendo re-lectura de archivos. La causa raiz es el **alto consumo de tokens por skill**: 27 skills suman 401 KB de prompts (~100K tokens), y cada agente invoca 6-8 skills por issue, consumiendo ~65% del contexto solo en instrucciones.

Propuesta: Migrar tareas mecanicas (validaciones, checks, builds) de skills-prompt a scripts Node.js/bash, reduciendo el consumo de contexto en ~40% sin perder funcionalidad.

401 KB	~100K	65%	40%
Skills totales (27 prompts)	Tokens en skills	Contexto en overhead	Reduccion posible

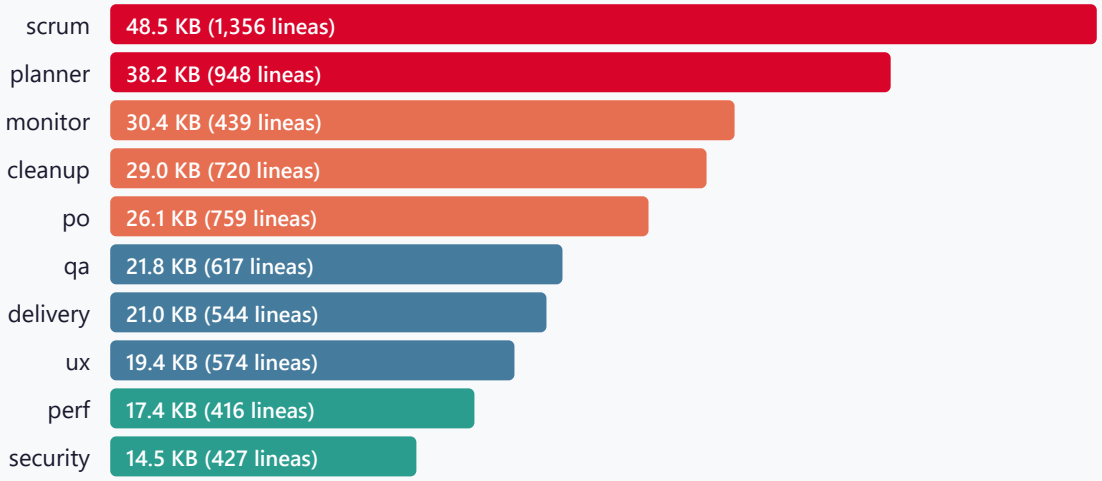
1. Diagnostico: Por que se compacta tanto

Claude Code tiene un contexto de ~200K tokens. Cada sesion de agente consume contexto de estas fuentes:

Fuente	Tamano	Tokens est.	Carga
CLAUDE.md (instrucciones del proyecto)	10.6 KB	~2,700	Siempre
Memory files (36 archivos .md)	~50 KB	~12,500	Siempre (indice + relevantes)
System prompt + tools definitions	~30 KB	~7,500	Siempre (framework)
Skills invocados (6-8 por agente)	**150-200 KB**	**~50,000**	**Cada invocacion de skill**
Tool results (git, builds, tests)	Variable	~30,000+	Acumulativo
TOTAL TIPICO	**~350 KB**	**~100K+**	**Supera 50% del contexto antes de codear**

Hallazgo clave: Un agente que invoca el pipeline completo (`/ops` + `/po` + `/backend-dev` + `/tester` + `/security` + `/review` + `/delivery`) consume ~50,000 tokens solo en cargar los prompts de esos 7 skills. Esto es **25% del contexto total**, antes de escribir una sola linea de codigo. Sumando CLAUDE.md, memory, y resultados de herramientas, el agente llega al 65% de consumo en overhead.

2. Mapa de consumo: Skills por tamano



Los top 5 skills (`scrum` , `planner` , `monitor` , `cleanup` , `po`) suman 173 KB — el **43% del peso total** de todos los skills.

3. Pipeline de un agente tipico: donde van los tokens

```
Pipeline completo — Issue M (feature media, ~60 min): FASE 0 /ops 10 KB — Verificar JAVA_HOME, node, disco, hooks FASE 1 /po 26 KB — Revisar criterios de aceptacion FASE 2 /backend-dev 12 KB — Implementar cambios FASE 3 /tester 11 KB — Ejecutar tests + cobertura /security 15 KB — Audit OWASP del diff FASE 4 /review 8 KB — Code review automatizado FASE 5 /delivery 21 KB — Commit + push + PR + merge ──────── SUBTOTAL SKILLS: 103 KB (~26K tokens) + CLAUDE.md: 11 KB (~3K tokens) + Memory: 50 KB (~13K tokens) + System prompt: 30 KB (~8K tokens) ═══════════════════════ OVERHEAD TOTAL: 194 KB (~50K tokens) = 25% del contexto + Tool results (git, build, tests): ~60K tokens acumulados + Codigo generado y editado: ~30K tokens ═══════════════════════ TOTAL SESION: ~140K tokens → COMPACTACION a los ~40 min
```

Evidencia real: En SPR-033, los 3 agentes murieron despues de 100+ turnos cada uno. El agente #1580 (shared-dtos) acumulo 269 mensajes antes de morir — claro indicador de agotamiento de contexto seguido de compactacion + degradacion.

4. Analisis: Que es overhead mecanico vs trabajo inteligente

La clave para optimizar es separar **tareas que requieren razonamiento de Claude** (y justifican consumir contexto) de **tareas mecanicas** que pueden ejecutarse como scripts sin consumir tokens:

Tarea	Skill actual	Tokens consumidos	Requiere razonamiento?	Migracion
Verificar JAVA_HOME, Node, disco	/ops	~2,500	No	Script Node.js
Verificar hooks existen	/ops	~1,500	No	Script Node.js
Ejecutar ./gradlew check	/builder	~2,000	No	Script bash
Buscar .env/secrets en diff	/security (parcial)	~2,000	No	Script Node.js
Ejecutar tests + parsear resultados	/tester (parcial)	~2,000	No	Script Node.js
git commit + push + crear PR	/delivery (parcial)	~3,000	No	Script Node.js
Limpiar worktrees, logs, procesos	/cleanup	~7,000	No	Script Node.js
Merge PR + delete branch	/delivery (parcial)	~1,500	No	Script bash
Analizar criterios de aceptacion	/po	~6,500	**Si**	Mantener como skill
Code review del diff	/review	~2,000	**Si**	Mantener como skill
Decidir arquitectura/patrones	/guru, /backend-dev	~5,000	**Si**	Mantener como skill
Escribir e implementar codigo	Dev skills	~3,000	**Si**	Mantener como skill
Planificar sprint/priorizar	/planner	~9,000	**Si**	Mantener como skill

~22K
Tokens migrables a scripts

~26K
Tokens que requieren Claude

46%
Overhead eliminable

0
Funcionalidad perdida

5. Propuesta de optimizacion: Script Pipeline

La propuesta central es crear un **pipeline de scripts** que ejecute las tareas mecanicas *antes y despues* de invocar a Claude, reduciendo dramaticamente los tokens consumidos:

Hoy (todo via prompts)

```
Claude inicia sesion ↓ carga CLAUDE.md (3K tok) ↓ carga Memory (13K tok) ↓ invoca /ops (2.5K tok) ↓ invoca /po (6.5K tok) ↓ invoca /backend-dev (3K tok) ↓ CODEA (variable) ↓ invoca /tester (3K tok) ↓ invoca /security (3.5K tok) ↓ invoca /review (2K tok) ↓ invoca /delivery (5K tok) ↓ invoca /cleanup (7K tok) ═══════════════ OVERHEAD: ~49K tokens COMPACTACIONES: 2-3 por sesion
```

Propuesto (scripts + prompts)

```
agent-runner.js (pre-Claude): ✓ ops-check.js (ya existe) ✓ validate-env.js (0 tokens) ✓ git branch setup (0 tokens) Claude inicia sesion ↓ carga CLAUDE.md (3K tok) ↓ carga Memory (13K tok) ↓ invoca /po (6.5K tok) ↓ invoca /dev skill (3K tok) ↓ CODEA (variable) ↓ invoca /review (2K tok) agent-runner.js (post-Claude): ✓ run-tests.js (0 tokens) ✓ security-scan.js (0 tokens) ✓ delivery.js (0 tokens) ✓ cleanup.js (0 tokens) ═══════════════ OVERHEAD: ~28K tokens (-43%) COMPACTACIONES: 0-1 por sesion
```

> **Principio:** Claude debe hacer lo que solo Claude puede hacer — razonar, analizar, decidir, codear. Todo lo demas (checks, builds, commits, merges, cleanup) debe ser un script que se ejecuta *fuera* del contexto.

6. Plan de migracion: 7 scripts propuestos

#	Script	Reemplaza	Tokens ahorrados	Prioridad	Esfuerzo
1	`agent-runner.js`	Orquestador pre/post Claude	Indirecto (habilita los demas)	P0	M (8-12h)
2	`pre-flight.js`	/ops (verificacion de entorno)	~2,500/agente	P0	S (2-4h)
3	`run-tests.js`	/tester (ejecucion de tests)	~3,000/agente	P0	S (2-4h)
4	`security-scan.js`	/security (scan mecanico)	~3,500/agente	P1	S (2-3h)
5	`auto-delivery.js`	/delivery (commit+PR+merge)	~5,000/agente	P1	M (6-8h)
6	`auto-cleanup.js`	/cleanup	~7,000/agente	P2	S (ya existe parcialmente)

7	`build-check.js`	/builder	~2,000/agente	P2	S (1-2h)

> **Ahorro total estimado:** ~23,000 tokens por agente por sesion. Con 3 agentes concurrentes, son ~69,000 tokens ahorrados por sprint. Equivale a eliminar ~1.5 compactaciones por agente.

7. Arquitectura propuesta: agent-runner.js

El cambio arquitectonico central es envolver la sesion de Claude en un script orquestador:

```
agent-runner.js — Ciclo de vida de un agente:
┌──────────────────────────────────────────────────────────────┐ │ PRE-CLAUDE (scripts, 0
tokens) │ │ │ │ 1. pre-flight.js │ │ ├── Verificar JAVA_HOME, Node, gh CLI │ │ ├──
Verificar disco libre > 2 GB │ │ ├── Verificar hooks presentes │ │ └── RESULTADO: env-
check.json (pass/fail) │ │ │ │ 2. git setup │ │ ├── git fetch origin main │ │ ├── git
checkout -b agent/N-slug origin/main │ │ └── Worktree ya listo │ │ │ │ 3. Leer issue de
GitHub │ │ ├── gh issue view N --json body,title,labels │ │ └── Guardar en /tmp/issue-
context.json │ ├──────────────────────────────────────────────────────────────┤ │ CLAUDE
(prompt minimo, solo razonamiento) │ │ │ │ Prompt: "Leer /tmp/issue-context.json. │ │
Invocar /po para criterios de aceptacion. │ │ Implementar los cambios requeridos. │ │
Invocar /review para validar el diff. │ │ Al terminar, escribir /tmp/agent-done.json │ │
con {files_changed, summary, pr_title, pr_body}" │ │ │ │ Skills usados: /po + /dev +
/review (solo 3, no 7+) │ │ Tokens ahorrados: ~23K vs pipeline actual │
├──────────────────────────────────────────────────────────────┤ │ POST-CLAUDE (scripts, 0
tokens — con emision de roles) │ │ │ │ 4. run-tests.js │ │ ├── emit("Claude", "Tester") ←
dashboard ve el nodo │ │ ├── ./gradlew check │ │ ├── Parsear resultados XML │ │ └── Si
falla: re-invocar Claude con errores │ │ │ │ 5. security-scan.js │ │ ├── emit("Tester",
"Security") ← transicion visible │ │ ├── Grep .env, secrets, tokens en diff │ │ └── Si
detecta: BLOQUEAR delivery │ │ │ │ 6. auto-delivery.js │ │ ├── emit("Security",
"DeliveryManager") ← rol visible │ │ ├── git add + commit (leer /tmp/agent-done.json) │ │
├── git push -u origin branch │ │ ├── gh pr create │ │ ├── gh pr merge --squash --delete-
branch │ │ └── Notificar via Telegram │ │ │ │ 7. auto-cleanup.js (si ultimo agente del
sprint) │ │ ├── Limpiar worktree │ │ ├── Podar branches │ │ └── Rotar logs │ │ │ │ * Todos
los scripts usan emit-transition.js para registrar │ │ su rol en la sesion → el dashboard
muestra el flujo │ │ completo con nodos e interacciones, identico a antes. │
└──────────────────────────────────────────────────────────────┘
```

8. Comparacion: antes vs despues

Metrica	Actual (todo prompt)	Propuesto (scripts + prompt)	Mejora
Skills invocados por agente	6-8	2-3	-60%
Tokens en overhead/sesion	~50K	~28K	-44%
Compactaciones por sesion	2-3	0-1	-66%
Tiempo perdido en re-lectura	~15 min/agente	~3 min/agente	-80%
Skills que requieren Claude	Todos (27)	Solo razonamiento (15)	-44% carga
Funcionalidad perdida	—		**CERO**

9. Premisa inamovible: sin perdida de funcionalidad

Garantia: Cada script propuesto implementa *exactamente* la misma logica que el skill que reemplaza. La diferencia es que se ejecuta *fuera* del contexto de Claude, sin consumir tokens.

Validacion: Cada migracion se verifica con:

- Test de paridad: skill vs script producen el mismo resultado
- Dry-run en paralelo: ambos corren, se comparan outputs
- Rollback inmediato: si el script falla, se invoca el skill original

9.1. Requisito critico: preservar visibilidad de roles en el dashboard

Regla: Cuando un skill se migra a script, el **rol del agente debe seguir apareciendo en el dashboard** con sus nodos y transiciones, exactamente como si el skill siguiera siendo invocado por Claude.

El dashboard muestra el "Flujo de agentes" como un grafo de nodos (roles) y transiciones (interacciones entre roles). Por ejemplo: `BackendDev → Tester → DeliveryManager`. Si `/tester` pasa a ser un script, el nodo "Tester" y sus transiciones **no pueden desaparecer** del flujo.

Mecanismo: emision de transiciones desde scripts

Hoy, el hook `activity-logger.js` captura las transiciones automaticamente cuando Claude invoca un skill (detecta el tool call `Skill` y registra `{from, to, ts}` en `agent_transitions[]` de la sesion). Cuando la tarea pasa a ser un script externo, ese hook ya no se dispara.

Solucion: Cada script que reemplaza un skill debe emitir su propia transicion escribiendo directamente en el archivo de sesion activa. Esto se logra con una funcion utilitaria compartida:

```
emit-transition.js — Modulo utilitario para scripts: // Uso desde cualquier script: // const {
emitTransition } = require('./emit-transition'); // emitTransition('BackendDev', 'Tester'); // al
iniciar // emitTransition('Tester', 'Security'); // al pasar al siguiente // Internamente: // 1.
Lee la sesion activa desde .claude/sessions/ // 2. Agrega {from, to, ts} a agent_transitions[] //
3. Agrega el rol a skills_invoked[] // 4. Escribe de vuelta al archivo de sesion // 5. El dashboard
lo lee en el proximo poll → nodo visible
```

Este modulo se implementa **una sola vez** y lo usan todos los scripts migrados.

Comparacion: visibilidad antes vs despues

Aspecto	Hoy (skill de Claude)	Propuesto (script + emit)	Diferencia
Nodo "Tester" en el flujo	✓ Aparece (via activity-logger hook)	✓ Aparece (via emit-transition.js)	Ninguna
Transicion BackendDev → Tester	✓ Registrada automaticamente	✓ Registrada por el script	Ninguna
Icono y color del rol	✓ Mapeado en AGENT_ICON_MAP	✓ Mismo mapa, mismo icono	Ninguna
Duracion del paso	✓ Calculada entre transiciones	✓ Mismas timestamps	Ninguna
Estado (✓/►/☐)	✓ Inferido de la sesion	✓ Inferido igual	Ninguna
Tokens consumidos	~3,000-7,000 por skill	**0**	-100%

> **Resultado:** El dashboard muestra *exactamente* los mismos nodos, transiciones, iconos y estados que antes. La unica diferencia es que los tokens se ahorran porque el trabajo lo hace un script en lugar de Claude leyendo un prompt de 20 KB.

Diagrama: flujo de datos con scripts

```
Hoy: Claude invoca /tester → activity-logger hook captura → escribe agent_transitions → dashboard
lee Propuesto: agent-runner.js ejecuta run-tests.js → run-tests.js llama emit-transition.js →
escribe agent_transitions → dashboard lee ↑ mismo archivo, mismo formato
```

Contratos de cada script migrado

Cada script que reemplaza un skill **debe** cumplir estos 3 contratos:

#	Contrato	Implementacion
1	**Emitir transicion de entrada**	`emitTransition(prevRole, thisRole)` al iniciar
2	**Emitir transicion de salida**	`emitTransition(thisRole, nextRole)` al terminar
3	**Registrar resultado en la sesion**	Escribir `skills_invoked[]` + `result: pass/fail` en el JSON de sesion

> **Ejemplo concreto:** `run-tests.js` reemplaza a `/tester`:
> 1. Emite `emitTransition("BackendDev", "Tester")`
> 2. Ejecuta `./gradlew check` y parsea resultados
> 3. Emite `emitTransition("Tester", "Security")`
> 4. En el dashboard: nodo "Tester" aparece con icono 🧪, conectado a BackendDev y Security — **identico a antes**

10. Otras mejoras complementarias

Mejora	Impacto	Esfuerzo	Detalle
Comprimir skills grandes	-15% tokens	S	scrum (48 KB) y planner (38 KB) tienen mucho texto redundante y ejemplos repetidos
Lazy loading de skills	-20% tokens	M	No cargar skill completo hasta que se invoque (requiere cambio en Claude Code)
Cache de resultados de tests	-5 min/agente	M	Si el diff no toca archivos de test, reutilizar resultado previo
Paralelizar gates	-10% tiempo	S	/security y /tester pueden correr en paralelo (no dependen entre si)
Prompt minimo para agentes	-30% tokens	S	En lugar de instrucciones detalladas en el prompt, referir a archivos locales que el agente lee bajo demanda

11. Roadmap de implementacion

Fase	Entregable	Descripcion
Fase 1 Quick wins	pre-flight.js + run-tests.js	Extraer /ops y /tester (parte mecanica) a scripts. Ahorro inmediato: ~5.5K tokens/agente. Sin cambios en Start-Agente.ps1.
Fase 2 Core pipeline	agent-runner.js + auto-delivery.js	Orquestador pre/post-Claude. Modifica Start-Agente.ps1 para usar agent-runner.js como wrapper. Ahorro: ~10K tokens/agente.

Fase 3 Full migration	security-scan.js + build-check.js + auto-cleanup.js	Completar migracion de todas las tareas mecanicas. Ahorro total: ~23K tokens/agente.
Fase 4 Optimizacion	Comprimir skills + prompts minimos	Reducir tamano de skills que quedan como prompt. Optimizar CLAUDE.md y memory files.

12. Metricas de exito

0-1
Compactaciones por sesion (hoy: 2-3)

28K
Tokens overhead (hoy: 50K)

3
Skills por agente (hoy: 7+)

100%
Funcionalidad preservada

Generado por Claude Opus 4.6 — Intrale Platform | 2026-03-16